PipelineHalt: Halting on Paper–Lean Drift and Asking Instead of Guessing

Benjamin Stanley Frohman
frohmanbenjamin@gmail.com
ORCID: 0009-0006-7068-3718

November 2026

Abstract

A formalization is only as faithful as the match between the statements it proves and the claims the accompanying paper makes. These two artifacts drift: indices shift, hypotheses are added, sets are re-encoded, numbering diverges. We describe PipelineHalt, a small open-source tool that treats the paper's claims as an anchor and the Lean statements as the live artifact, flags places where the Lean is *silent*, *narrower*, or *conditional* relative to the paper, or where the paper itself is *ambiguous*, and then halts with a precise clarification question rather than correcting anything. Answers are recorded verbatim in the anchor under stable identifiers so questions are not re-asked. The tool does not and cannot recover intent that was never written down; its matching is heuristic and syntactic. We report its real output on a small Lean development, including a false positive.

1 Introduction

Machine-checked proofs in Lean 4 [1] with Mathlib [5] certify that a *Lean statement* holds. Readers, however, consume the *paper*. Whether the paper's Theorem 3.1 and the Lean `two_le_gap` say the same thing is a question the kernel does not ask. Large-language-model agents that write or revise either artifact make the problem sharper: they will happily "fix" a mismatch in whichever direction is locally convenient.

PipelineHalt is an extension of RecurringAlignment [3]. Its design principle is simple: **on a detected difference, stop and ask the author**. It never edits a claim or a Lean file.

2 The residual-gap problem

Call the *residual gap* the set of differences between what the paper asserts and what the Lean proves that survive compilation. Some are benign conventions (0- vs. 1-indexed primes); some change the mathematics (an extra hypothesis). Syntactically the two cases often look identical, and distinguishing them requires information that lives only in the author's head. A tool can therefore at best *surface* the gap and *elicit* a decision; it cannot close it alone. We make that limitation the interface.

3 Design

Inputs. The anchor is a list of claims (id, printed number, title, statement, optional explicit `lean:` mapping and stated assumptions), read from JSON, Markdown, or LaTeX theorem environments (with amsthm section numbering reproduced); PDF is supported best-effort via `pdftotext`. Lean files are parsed with a lightweight bracket-aware scanner extracting declarations, binders, type, docstring, `sorry`, and use of project axioms.

Mapping file. The practical way to cut first-run noise is a small, hand-written *mapping file*, `pipelinehalt.map.json`, kept separate from the extracted claims. It maps each claim id to Lean declaration names, optionally lists *aliases* (e.g. a docstring's "Theorem 1" for the paper's Theorem 3.1), and may list declarations deliberately left unmapped. It is found by default beside the anchor or in a parent directory up to the repository root, and can be overridden (`--map`) or disabled (`--no-map`). Mapped claims skip heuristic pairing, and aliases suppress numbering questions; all other checks still run on the mapped declarations. The mapping is human-authored: it says *where to look*, reduces pairing noise, and is not validated semantically. A wrong mapping yields wrong findings.

Matching. Without a mapping, each declaration is assigned to the claim maximizing $0.6\,D(\text{sig}) + 0.4\,D(\text{doc}) + 0.3\,[\text{doc cites the claim's number}]$, where D is the Dice coefficient [2] over stemmed tokens; definitions match definition-like claims only. This is a heuristic and is the main source of error.

Checks. Per claim:

- **Silent**: a mapped name is missing, or nothing matches.
- **Narrower**: a Lean lower bound on a variable differs from the claim's, or a set literal in the claim appears in no matched statement.
- **Conditional**: a hypothesis binder or premise not covered by the claim text or its stated assumptions; `sorry`; a project axiom.
- **Ambiguous**: hedging words, empty statement, title unrelated to statement, duplicate ids, or a docstring theorem number disagreeing with the printed one.

Each mismatch has an id $= \text{sha256}(\text{claim}, \text{kind}, \text{decl}, \text{key})$ truncated to 12 hex digits, and a weight (silent 1, narrower 0.6, conditional 0.5, sorry 0.8, axiom 0.7, ambiguous 0.3); claim drift is the capped sum.

4 Clarification protocol

Unanswered mismatches yield status HALT (exit code 2). Interactively, questions are asked on the terminal; non-interactively, they are written to a pending file to be filled in and imported. An answer is stored verbatim with a verdict parsed from its first word: *intentional* (weight 0), *unintended* (full weight), or other (half weight). Since ids are stable, an answered question is never re-asked; if the Lean changes so that the key changes, a new question appears. For agents, a JSON protocol (`pipelinehalt/1`) returns the questions and accepts answers; the accompanying prompt template instructs agents to relay questions unchanged and never answer them. A CI mode emits GitHub annotations and fails on halts or drift above a threshold.

5 Example: DHL[3,2] and prime gaps

We ran PipelineHalt 0.4.0 on the repository for [4] (commit `69a13d3`): 9 claims extracted verbatim from `paper/DHL32_note.tex`, 34 Lean declarations. Table 1 compares a run without a mapping with a run using the mapping file shipped in `examples/dhl32/`, which we drafted from the Lean docstrings.

Run	Status	Drift	Open questions	Unmatched-citation notes
No mapping (`--no-map`)	HALT	0.267	5	3
Mapping file (default)	HALT	0.160	3	0

Table 1: Real output on the DHL[3,2] development.

Without a mapping. The heuristic paired `Ptoy_DHL32` with Theorem 3.2. This produced a *false* "silent" finding for Theorem 3.3 and a spurious unstated-hypothesis question (`H.card = 3`) from a wrongly paired declaration. It also asked three docstring-numbering questions ("Theorem 1/2/4"). It *missed* the $n \geq 1$ vs. $n \geq 2$ difference, because `two_le_gap` was not paired with Theorem 3.1.

With the mapping file. Three questions remain:

- Theorem 3.1: `two_le_gap` and `gap_even` assume $1 \leq n$, while the paper says $n \geq 2$ (two questions);
- Theorem 3.3: the paper's $\{0, 1\}$ appears in no mapped Lean statement.

The second is very likely a false positive: `Ptoy` is defined by `n % 4 < 2`, which is equivalent to $n \bmod 4 \in \{0, 1\}$ over $\mathbb{N}$, but a syntactic check cannot establish that. The first may be an indexing convention; only the author can say. The mapping removed pairing and numbering noise; it did not, and cannot, resolve these. They remain unanswered in the published example; we did not answer them on the author's behalf.

6 Limitations

PipelineHalt is syntactic. It does not understand mathematics, does not check proofs, and does not recover unstated intent; an aligned report means only that these checks found nothing. Token matching mis-assigns declarations unless a mapping file is supplied, and the mapping itself is unverified; bound and set checks miss equivalent encodings and miss differences expressed otherwise; LaTeX macro normalization is partial; PDF extraction is fragile. Drift weights are arbitrary conventions, not measurements. The evaluation is a single small example.

7 Conclusion

PipelineHalt turns paper–Lean drift into explicit, recorded questions with stable ids, and refuses to resolve them itself. The value lies not in the detection heuristics, which are modest, but in the discipline: differences are surfaced, the pipeline halts, and the author's answer becomes part of the record.

Availability. Source, tests and example outputs: `https://github.com/BenFrohman/PipelineHalt` (code MIT; paper CC BY 4.0), derived from `packages/pipelinehalt` in [3].

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.

[2] Lee R. Dice. Measures of the amount of ecologic association between species. *Ecology*, 26(3):297–302, 1945.

[3] Benjamin Stanley Frohman. RecurringAlignment: recurring alignment checks for agent pipelines. `https://github.com/BenFrohman/RecurringAlignment`, 2026.

[4] Benjamin Stanley Frohman. Resolving a category error: DHL[3,2] yields $H_1 \leq 6$, not $H_1 = 2$. `https://github.com/BenFrohman/DHL-3-2-implies-liminf-prime-gap-le-6`, 2026.

[5] The mathlib Community. The Lean mathematical library. In *Proceedings of CPP 2020*, pages 367–381. ACM, 2020.